EJADA EGYPT SUMMER INTERNSHIP PROGRAM 2026

Cloud Build Track — OCI & Terraform

Week 3 — Lab 3

Terraform Implementation & Evidence Report

Containerized NGINX Workload on Oracle Kubernetes Engine — VCN-Native Networking, a Managed Node Pool, and CSI-Backed Block Storage

Student	Salah Abdelhady
Tenancy	ociejada
Region	me-jeddah-1
Compartment	intern-18-salah-abdelhady-cmp
Working stack	terraform/nonprd/week3-containerized-oke/

Contents

1. Objectives

This lab rebuilds the Week 3 workload entirely through Terraform, without a manual Console-first pass. The stack provisions the network, the OKE cluster, and a containerized NGINX workload backed by block storage, then is destroyed cleanly once the evidence below was captured.

1. Deploy the Week 3 environment 100% with Terraform: a VCN on 10.2.0.0/16, three subnets, and an OKE cluster.
2. Network module — VCN, Internet Gateway, NAT Gateway, Service Gateway, and a shared log group (w3-vcn, w3-igw, w3-nat, w3-sgw, w3-vcn-logs).
3. Subnet module — subnet, route table, security list, and optional flow logs, instantiated three times for lb, workers, and pods.
4. OKE module — a cluster with VCN-native pod networking (OCI_VCN_IP_NATIVE), a managed worker node pool, and dedicated API, worker, and pod NSGs.
5. Once the cluster reached ACTIVE, an NGINX Deployment, a CSI-backed block volume PVC (oci-bv, mounted at /data), and a Service of type LoadBalancer.
6. A clean destroy after evidence capture, leaving no billable resources behind.

Building blocks delivered

Building block	What landed
Network related-group	module.network — w3-vcn, Internet Gateway, NAT Gateway, Service Gateway, log group w3-vcn-logs
Subnet related-group	module.subnet × 3 (w3-lb-subnet, w3-workers-subnet, w3-pods-subnet)
OKE related-group	Cluster w3-oke ACTIVE, node pool w3-np (size 1), CNI OCI_VCN_IP_NATIVE
Application layer (PVC / Service LB)	kubectl-managed namespace w3-demo, PVC Bound 50Gi, Service VIP 193.122.68.107
Destroy	36 Terraform resources destroyed; Kubernetes Service, PVC, and namespace removed first

2. Environment

Item	Value
Tenancy	ociejada
Compartment	intern-18-salah-abdelhady-cmp
Region	me-jeddah-1
Availability domain	oXVt:ME-JEDDAH-1-AD-1
Terraform	v1.15.8 (windows_amd64)
Provider	oracle/oci v6.37.0 (~> 6.0)
Authentication	OCI config profile DEFAULT
VCN	w3-vcn — 10.2.0.0/16
Public load balancer subnet	w3-lb-subnet — 10.2.1.0/24
Private workers subnet	w3-workers-subnet — 10.2.10.0/24
Private pods subnet	w3-pods-subnet — 10.2.128.0/18

Item	Value
Kubernetes services CIDR	10.96.0.0/16 (outside the VCN)
Tags	Project=Ejada-Cloud-Build, Lab=week3, ManagedBy=Terraform

Runtime values captured during the lab

The cluster and workload below were live for the duration of validation and were destroyed afterward; the values are retained here as evidence.

Item	Value
Cluster	w3-oke — ACTIVE, BASIC, Kubernetes v1.34.1, CNI OCI_VCN_IP_NATIVE
Cluster OCID	ocid1.cluster.oc1.me-jeddah-1.aaaaaaaavhl7onzm5htuw5k2reau64kdc7effu4itsy5qyztscilhoyslgma
Node pool	w3-np — size 1, VM.Standard.E4.Flex (1 OCPU / 8 GB)
Worker	Private IP 10.2.10.193, no public IP
Namespace	w3-demo
PVC	nginx-data — Bound, 50Gi, StorageClass oci-bv
Image	docker.io/library/nginx:alpine
Service load balancer	External IP 193.122.68.107
Volume mount	/data on the NGINX container

The Part 1 and Part 2 CLI snapshots reference different internal VCN OCIDs from the same apply-and-recreate cycle, though the display names and CIDRs are identical across both. The names, CIDRs, and cluster OCID recorded in this document are the values of record. No secrets, keys, or tokens are included.

3. Architecture

3.1 Target architecture (Terraform)

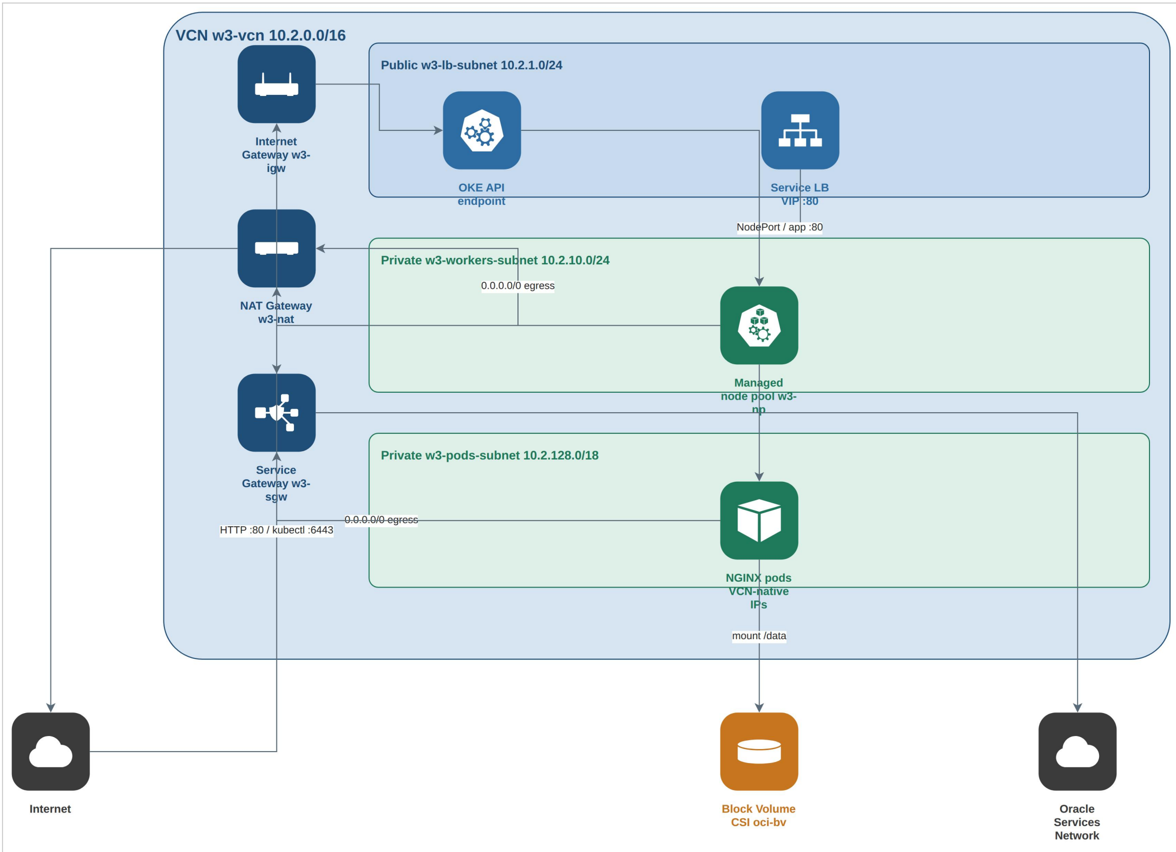


Figure 1. *Target network architecture — VCN, gateways, subnets, and the OKE workload path.*

The Internet-facing path runs through the Internet Gateway into the public load balancer subnet, which carries both the OKE API endpoint and the Kubernetes Service load balancer. Workers and pods stay fully private, with no public IP addresses assigned. The NAT Gateway covers general egress for workers and pods, while the Service Gateway reaches the Oracle Services Network for the CSI driver and other regional APIs. Pods take VCN-native IP addresses from the 10.2.128.0/18 pods subnet rather than an overlay CIDR.

3.2 HTTP traffic path

Figure 2. *Request path from the browser through the Service load balancer to the pod, and the CSI volume mount.*

Clients never reach workers or pods directly. The Service of type LoadBalancer provisions an OCI Flexible Load Balancer in the public subnet, which forwards to the worker's NodePort and on to the container through kube-proxy. The persistent volume is an external Block Volume attached through the CSI driver (oci-bv), not File Storage. The historical VIP for this lab was 193.122.68.107, and the worker's private IP was 10.2.10.193.

3.3 Resource roles

Layer	Resources
Network module (network.tf → modules/network)	VCN, Internet Gateway, NAT Gateway, Service Gateway, shared log group w3-vcn-logs
Subnet module × 3 (subnets.tf)	Per key: subnet + route table + security list; the lb subnet also adds a flow log and capture filter
OKE module (oke.tf; NSGs in nsg.tf)	Cluster, managed node pool, NSGs (w3-nsg-oke-api / -workers / -pods)
Application layer (kubectl, not Terraform)	Namespace, PVC on oci-bv, Deployment, Service load balancer

4. Source Inventory

The root module lives at terraform/nonprd/week3-containerized-oke/ and calls three cohesive modules: modules/network, modules/subnet, and modules/oke. The recommended read order is versions.tf → providers.tf → variables.tf → locals.tf → network.tf → subnets.tf → oke.tf → outputs.tf. There is no data.tf: the All-OSN lookup lives inside modules/network, and the availability-domain and node-image catalogs are queried directly in oke.tf.

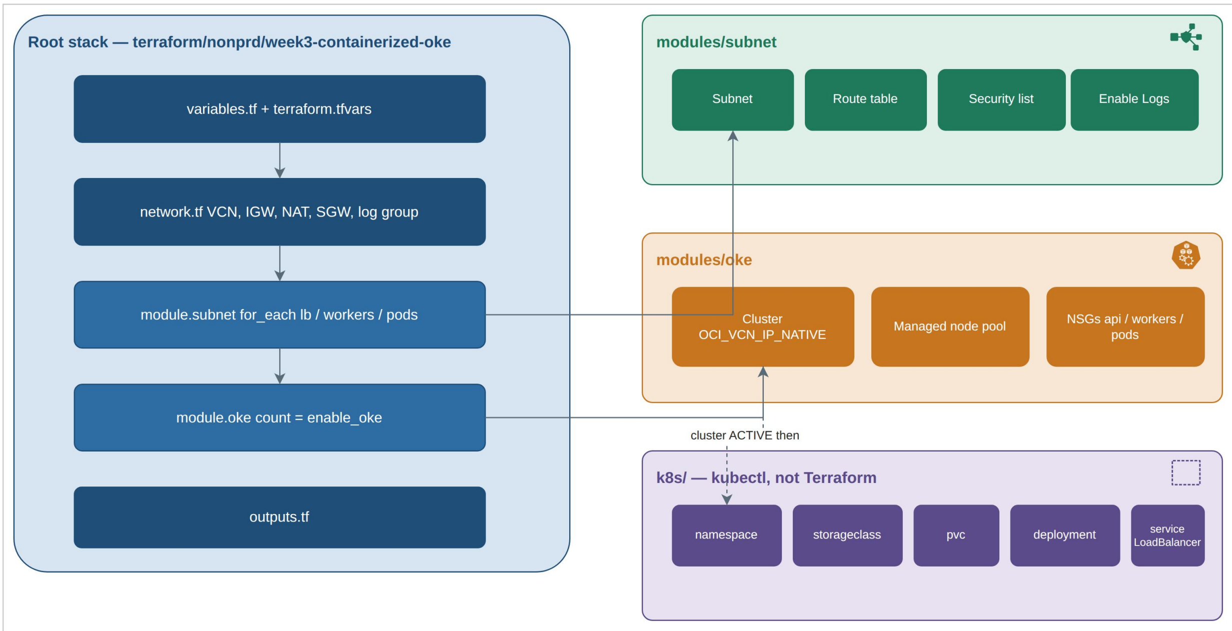


Figure 3. *Module map — root stack calling the subnet and OKE modules, with the application layer applied afterward.*

4.1 Root files

File	Purpose
versions.tf	Terraform >= 1.5; required provider oracle/oci ~> 6.0
providers.tf	OCI provider: region + config_file_profile; ignore_defined_tags for the intern-uneditable Oracle-Tags.CreatedBy / CreatedOn; optional API-key variables stay null under a CLI profile
variables.tf	All tunable arguments — compartment, CIDRs, names, shapes, OCPU/memory, ports, counts, tags, DNS labels, feature flags (enable_oke, enable_service_gateway, enable_oke_nsgs, enable_https_ingress), log retention, Kubernetes version, node image, SSH public key
terraform.tfvars.example	Committed template with placeholder values only; real values stay in the gitignored terraform.tfvars
locals.tf	Freeform tags; display names derived from the name prefix or overrides (w3-vcn, w3-oke, NSGs, …); availability-domain and OKE version/image selection; security-list and route-table rule payloads; the subnets map passed into module.subnet with gateway IDs sourced from module.network
network.tf	module.network (VCN, Internet Gateway, NAT Gateway, Service Gateway, shared log group); moved blocks map earlier root-level addresses into the module
subnets.tf	module.subnet for_each across lb, workers, and pods
oke.tf	module.oke with count = var.enable_oke ? 1 : 0; availability-domain and node-image data sources; wires subnet IDs and NSG names into the module

File	Purpose
outputs.tf	VCN and subnet IDs, Service Gateway, log group, cluster and node-pool IDs, CNI type, NSG IDs, and kubeconfig / kubectl-apply hints
.terraform.lock.hcl	Locked provider version 6.37.0
README.md	Stack operator notes; this document remains the authoritative narrative for the full lab

4.2 modules/network — VCN, gateways, shared log group

File	Purpose
main.tf	All-OSN data source; oci_core_vcn; Internet Gateway; NAT Gateway; Service Gateway (count when enable_service_gateway); shared oci_logging_log_group (count when any subnet has logging enabled)
variables.tf	Display names, VCN CIDR/DNS, Service Gateway and log-group flags, tags
outputs.tf	vcn_id, gateway IDs, log_group_id, osn_cidr
versions.tf	Inherits the root OCI provider

This is one related group, not a module per gateway. Subnets stay in modules/subnet, and cluster NSGs stay in modules/oke.

4.3 modules/subnet — subnet, route table, security list, logs

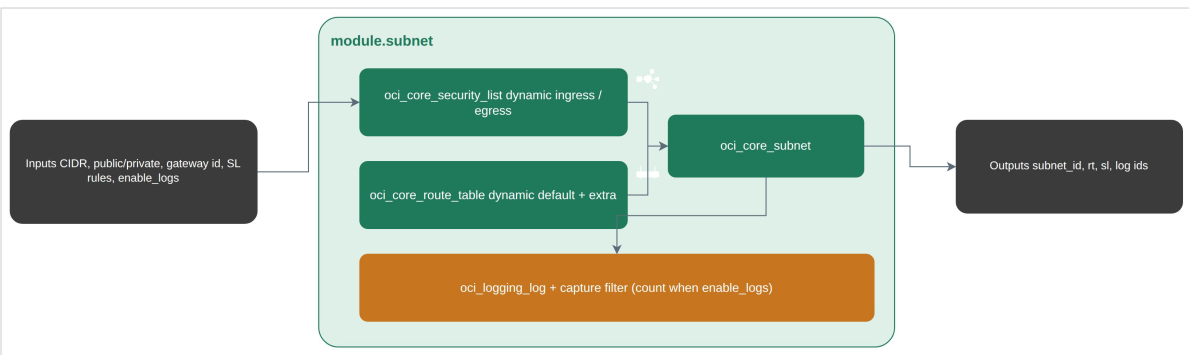


Figure 4. *Subnet module internals — one related group instantiated per subnet key.*

File	Purpose
main.tf	Security list (dynamic ingress/egress with nested TCP/UDP/ICMP options), route table (dynamic default 0.0.0.0/0 plus extra rules), subnet, and an optional capture filter and service flow log (count when enable_logs)
variables.tf	Display name, CIDR, DNS, public/private flag, gateway ID, extra routes, security-list rules, log flags (enable_logs, create_log_group, log_group_id)
outputs.tf	subnet_id, route-table and security-list IDs, log and capture-filter IDs
versions.tf	Inherits the root OCI provider

For this lab, create_log_group is false and log_group_id is sourced from module.network. Only the lb subnet has enable_logs = true. Workers and pods route their default 0.0.0.0/0 through NAT with a Service Gateway extra rule, while the lb subnet routes its default through the Internet Gateway.

4.4 modules/oke — cluster, VCN-native CNI, managed pool

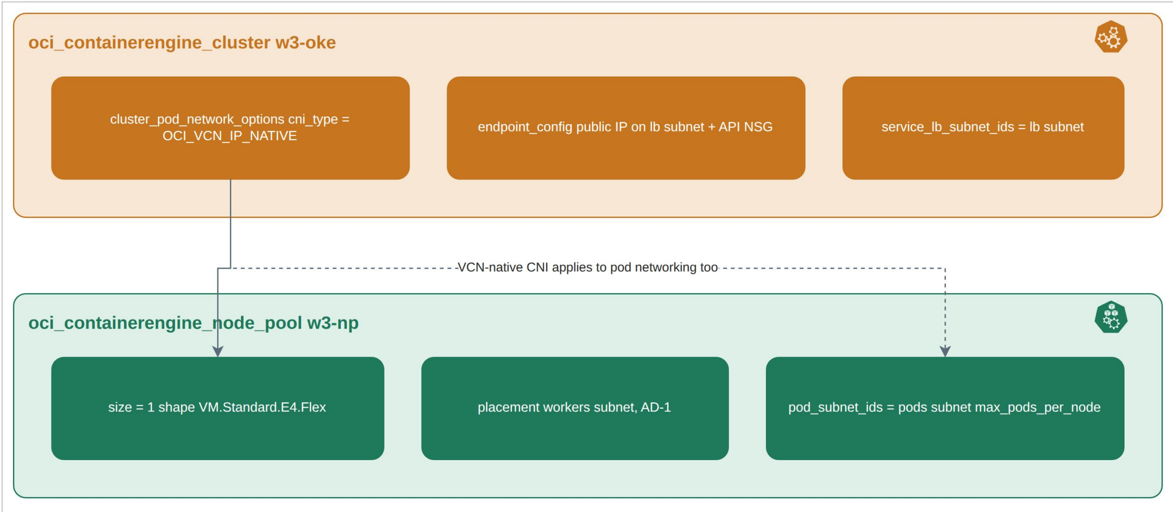


Figure 5. *OKE cluster and node pool, both configured for VCN-native pod networking.*

File	Purpose
main.tf	oci_containerengine_cluster with cni_type = OCI_VCN_IP_NATIVE; oci_containerengine_node_pool with the worker subnet, pod subnet IDs, and a Flex node_shape_config
nsg.tf	API, worker, and pod NSGs plus rules (count / for_each), attached to the API endpoint, workers, and VCN-native pod VNICs
variables.tf	Cluster name/type/version/CNI, endpoint/worker/pod/load-balancer subnet IDs, node shape/OCPU/memory/size/image, SSH key, NSG names, CIDRs, and ports
outputs.tf	Cluster and node-pool IDs, endpoints, CNI type, NSG IDs
versions.tf	Inherits the root OCI provider

NSGs live inside this module rather than as a root file or a one-resource wrapper. Kubernetes workloads themselves are not a Terraform module.

4.5 k8s/ — application layer after the cluster exists

File	Purpose
namespace.yaml	Namespace w3-demo
storageclass.yaml	Explicit oci-bv definition (blockvolume.csi.oraclecloud.com). The cluster already ships this class, so this manifest is kept for reference; see Section 10.2 for how a reapply of it was handled
pvc.yaml	PVC nginx-data, 50Gi, ReadWriteOnce, storageClassName: oci-bv
deployment.yaml	NGINX Deployment, image docker.io/library/nginx:alpine, volume mount at /data
service.yaml	Service type: LoadBalancer, Flexible shape 10–10 Mbps annotations
README.md	kubectl apply / delete order for this layer

Not committed: terraform.tfvars, terraform.tfstate, .terraform/, local plan binaries, and private keys.*

5. Variables and terraform.tfvars.example

Changeable arguments live in variables.tf. Environment-specific values live in the gitignored terraform.tfvars; the committed template is terraform.tfvars.example.

Variable	Role in this lab
compartment_ocid	intern-18 (placeholder value in the example file)
region / config_file_profile	me-jeddah-1 / DEFAULT
name_prefix / project_tag / lab_tag	w3 / Ejada-Cloud-Build / week3
enable_oke	false for the Part 1 network apply; true for the Part 2 cluster and node-pool apply
enable_service_gateway / enable_oke_nsgs	both true
vcn_cidr / subnets	10.2.0.0/16; keys lb, workers, pods with CIDRs, DNS labels, public/private flag, route_type, enable_logs
allowed_ssh_cidr	Laptop CIDR for API :6443 and worker SSH (placeholder x.x.x.x/32 in the example)
ssh_public_key	Node metadata; the example file shows a dummy ssh-ed25519 AAAA... line
oke_cluster_type / oke_cni_type	BASIC_CLUSTER / OCI_VCN_IP_NATIVE
oke_endpoint_public	true, to allow kubectl from a laptop
oke_services_cidr	10.96.0.0/16
oke_node_pool_size / shape / OCPU / memory	1 / VM.Standard.E4.Flex / 1 / 8 GB
k8s_namespace	w3-demo

Display names such as w3-vcn, w3-oke, and the NSG names are derived in locals.tf from name_prefix unless an override is set; root resources do not hardcode them.

Example (placeholder values, not the live tfvars)

```
region              = "me-jeddah-1"
config_file_profile = "DEFAULT"
compartment_ocid    = "ocid1.compartment.oc1..aaaaaaaaXXXXXXXX"
enable_oke          = false   # Part 1; flipped to true for Part 2
vcn_cidr            = "10.2.0.0/16"
oke_cni_type        = "OCI_VCN_IP_NATIVE"
oke_node_pool_size  = 1
k8s_namespace       = "w3-demo"
```

6. Terraform Design Standards Applied

The stack follows a consistent set of module-design conventions carried through the Cloud Build track: changeable values are isolated from structure, resources are grouped by relationship rather than wrapped one-to-one, and the application layer stays outside of Terraform state.

Principle	Implementation in this stack
Variables-driven configuration	Changeable arguments in variables.tf; environment values in tfvars; only the example file is committed
Related-group modules	network (VCN + Internet Gateway + NAT Gateway + Service Gateway + shared log group); subnet (subnet + route table + security list + logs) × 3; oke (cluster + native CNI + managed pool + NSGs in nsg.tf) — see Figure 3
No thin one-resource wrappers	Gateways live in modules/network; NSGs live in modules/oke; there is no one-resource wrapper per gateway or NSG
Multi-file root layout	versions.tf → providers.tf → variables.tf → locals.tf → network.tf → subnets.tf → oke.tf → outputs.tf
Application layer separated from infrastructure	k8s/ is applied with kubectl only after the cluster reaches ACTIVE
Documentation as a finished deliverable	This document is the single Terraform and evidence narrative for Week 3, covering code, commands, evidence, diagrams, and the destroy

7. Commands Executed (Successful Path)

Working directory: terraform/nonprd/week3-containerized-oke. Profile DEFAULT, region me-jeddah-1, Windows PowerShell.

7.1 Tooling baseline

```
terraform version
  → Terraform v1.15.8 + provider registry.terraform.io/oracle/oci v6.37.0
```

7.2 Part 1 — network (enable_oke = false)

Captured 2026-08-15, approximately 18:53 +03.

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
# Apply complete! Resources: 16 added, 0 changed, 0 destroyed.
# No oci_containerengine_*
```

The 16 managed resources were: the VCN, Internet Gateway, NAT Gateway, Service Gateway, log group, three subnet + route table + security list groups, and the lb subnet's capture filter and flow log.

7.3 Part 2 — OKE (enable_oke = true)

Captured 2026-08-16, approximately 00:24 +03, after the intern's create permission on the cluster-family / OKE service policy was granted (Section 10.1).

```
# terraform.tfvars: enable_oke = true, ssh_public_key set
terraform plan
terraform apply
# Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
# cluster w3-oke  (~8m40s)
# node pool w3-np (~5m26s)
```

Cluster OCID: `ocid1.cluster.oc1.me-jeddah-1.aaaaaaaavhl7onzm5htuw5k2reau64kdc7effu4itsy5qyztscilhoyslgma`

Node pool OCID: `ocid1.nodepool.oc1.me-jeddah-1.aaaaaaaamuhzibv3uogyvmnn753xsc5e4bbkt3s6furajrrelnmehg75g3sa`

The OKE NSGs were already present in the module's state; this apply added only the cluster and the node pool.

7.4 Workload deployment via kubectl

The request path exercised here is the one traced in Figure 2 (Section 3.2).

```
$env:KUBECONFIG = "$env:USERPROFILE\.kube\w3-oke"

oci ce cluster create-kubeconfig `
  --cluster-id ocid1.cluster.oc1.me-jeddah-1.aaaaaaaavhl7onzm5htuw5k2reau64kdc7effu4itsy5qyztscilhoyslgma `
  --file $env:USERPROFILE\.kube\w3-oke `
  --region me-jeddah-1 `
  --token-version 2.0.0 `
  --kube-endpoint PUBLIC_ENDPOINT
```

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/pvc.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

```
kubectl -n w3-demo get pods,deploy
  → pod/nginx-…  1/1  Running
  → deployment.apps/nginx  1/1

kubectl -n w3-demo get pvc nginx-data
  → Bound  50Gi  RWO  oci-bv  volume csi-082c7776-ed2e-4301-80e1-fd4d7cf02fe2

kubectl -n w3-demo get svc nginx
  → LoadBalancer  EXTERNAL-IP 193.122.68.107  80:31785/TCP

kubectl -n w3-demo exec deploy/nginx -c nginx -- df -h /data
  → /dev/sdb  48.9G  mounted on /data
```

Browser check: http://193.122.68.107/ returned the nginx welcome page.

7.5 Destroy

Captured 2026-08-16, approximately 01:18–01:26 +03. Kubernetes objects sit outside Terraform state, so they were removed first.

```
kubectl -n w3-demo delete svc nginx
kubectl -n w3-demo delete deploy nginx
kubectl -n w3-demo delete pvc nginx-data
kubectl delete ns w3-demo
terraform destroy -auto-approve -no-color
# Destroy complete! Resources: 36 destroyed.
# terraform state list → empty
```

8. Evidence Catalogue

The raw evidence files back the excerpts already quoted through this document; the substance is summarized below.

File	What it proves
terraform-part1-network-apply.txt	Part 1 apply added 16 resources; enable_oke = false; VCN 10.2.0.0/16
terraform-part1-outputs.txt	Part 1 outputs (subnet IDs, log group, Service Gateway); OKE outputs null
terraform-part1-state-list.txt	16 managed addresses; no oci_containerengine_*
oci-cli-part1-network-snapshot.txt	Live w3-* VCN, gateways, subnets, route tables, security lists, log group
compartment-intern-18-audit.txt	All oci_* resources use the intern-18 compartment OCID
terraform-part2-oke-apply.txt	Cluster and node pool added (2 resources); CNI OCI_VCN_IP_NATIVE; Kubernetes v1.34.1
terraform-part2-oke-outputs.txt	Cluster/node-pool IDs, NSG IDs, kubeconfig hint
terraform-part2-oke-state-list.txt	Cluster, node pool, NSGs, three subnet modules
oci-cli-part2-oke-snapshot.txt	w3-oke ACTIVE, w3-np ACTIVE size 1, three NSGs
terraform-part2-oke-destroy.txt	36 resources destroyed, exit 0, empty state; kubectl cleanup ran first
oci-cli-post-destroy-inventory.txt	intern-18 shows zero live billable resources; TERMINATED / DELETED ghosts only

8.1 Key excerpts

Part 1 state (abbreviated):

```
oci_core_vcn.this
oci_core_internet_gateway.this
oci_core_nat_gateway.this
oci_core_service_gateway.this[0]
oci_logging_log_group.vcn_flow[0]
module.subnet["lb"].oci_core_subnet.this
module.subnet["lb"].oci_logging_log.flow[0]
module.subnet["workers"].oci_core_subnet.this
module.subnet["pods"].oci_core_subnet.this
# no oci_containerengine_*
```

Part 2 OCI CLI (highlights):

```
w3-oke  ACTIVE  BASIC_CLUSTER  v1.34.1
        ocid1.cluster.oc1.me-jeddah-
1.aaaaaaaavhl7onzm5htuw5k2reau64kdc7effu4itsy5qyztscilhoyslgma
w3-np   ACTIVE  VM.Standard.E4.Flex  size=1
NSGs    w3-nsg-oke-api / w3-nsg-oke-workers / w3-nsg-oke-pods  AVAILABLE
```

Part 2 outputs (highlights):

```
oke_cluster_id     = ocid1.cluster.oc1.me-jeddah-
1.aaaaaaaavhl7onzm5htuw5k2reau64kdc7effu4itsy5qyztscilhoyslgma
oke_cni_type       = OCI_VCN_IP_NATIVE
kubernetes_version = v1.34.1
oke_node_pool_id   = ocid1.nodepool.oc1.me-jeddah-
1.aaaaaaaamuhzibv3uogyvmnn753xsc5e4bbkt3s6furajrrelnmehg75g3sa
```

Stage	Resource count
Part 1 apply added	16
Part 2 apply added	2 (cluster + node pool; NSGs already in state)
Destroy	36 destroyed

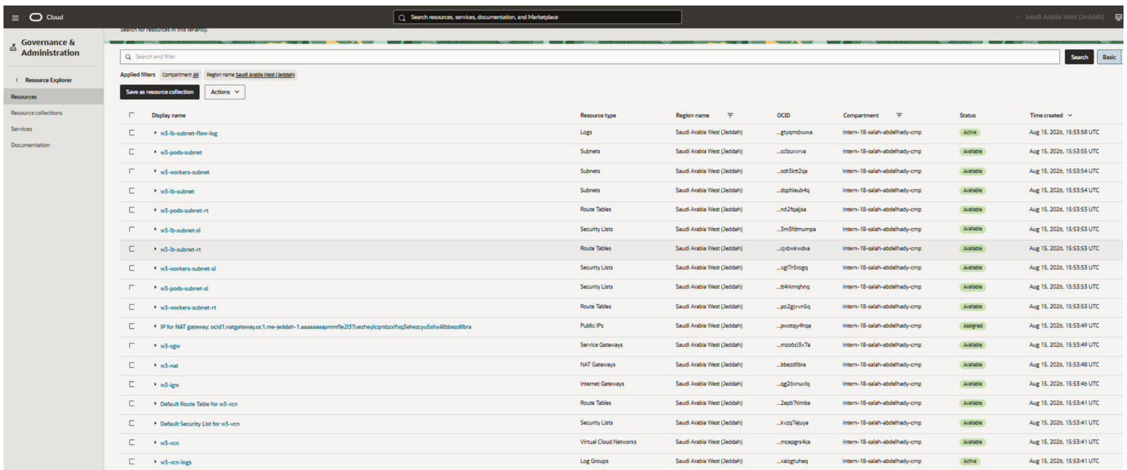
Workload validation: pods Running 1/1; PVC Bound 50Gi on oci-bv; the browser reached http://193.122.68.107/ and returned the nginx welcome page; df -h /data inside the pod showed /dev/sdb mounted with roughly 48.9G available.

9. Screenshot Evidence

The screenshots below were captured from the OCI Console and the kubectl CLI after each Terraform stage. Session tokens and kubeconfig tokens are cropped out of every image; no secrets, PEM paths, or terraform.tfvars content appear in any capture. A small number of inventory-list screenshots (a combined subnet list, a combined route-table list, and the VCN IP-administration view) were captured alongside these but are not repeated here, since every resource they show is already covered individually below. The node-pool worker is likewise covered by the node-pools tab (B02b) and the worker instance detail (B04) rather than a separate Nodes-tab capture.

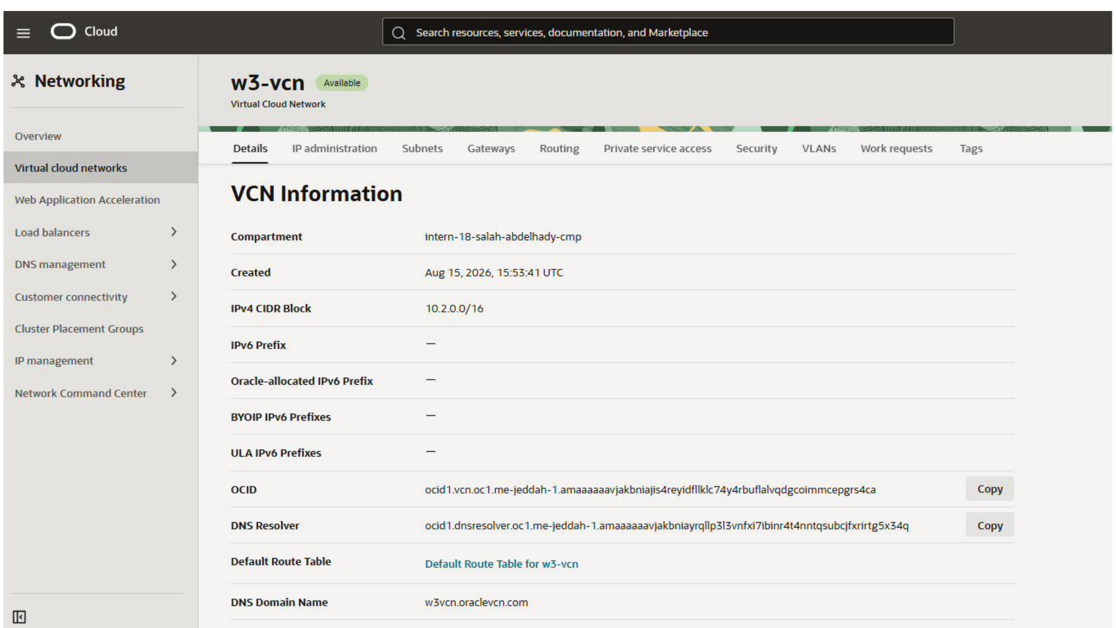
9.1 Stage A — network

A00 — Resource Explorer inventory



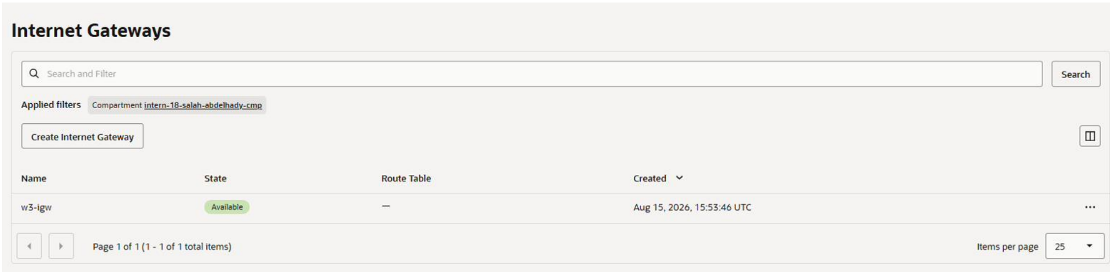
***A00* —** *Resource Explorer showing every w3-* resource as Available or Active.*

A01 — VCN w3-vcn

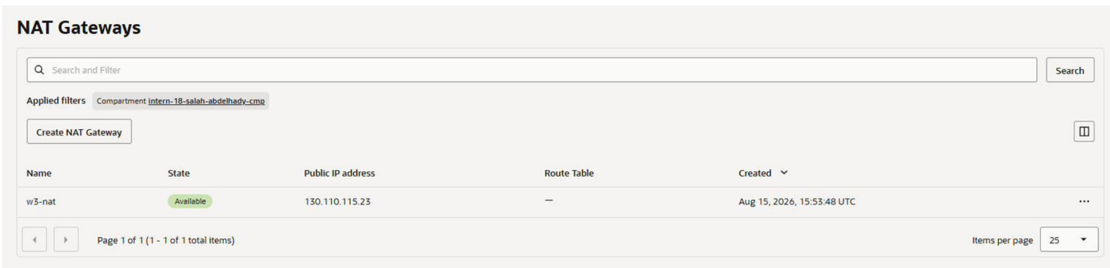


***A01* —** *VCN w3-vcn Available, CIDR 10.2.0.0/16.*

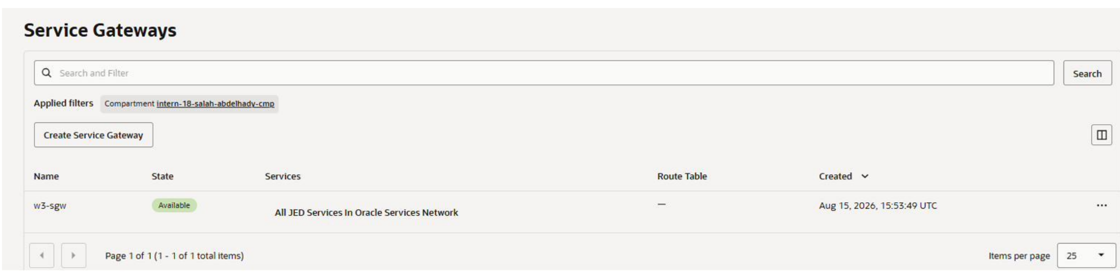
A02 — gateways



A02* — *Internet Gateway w3-igw Available.

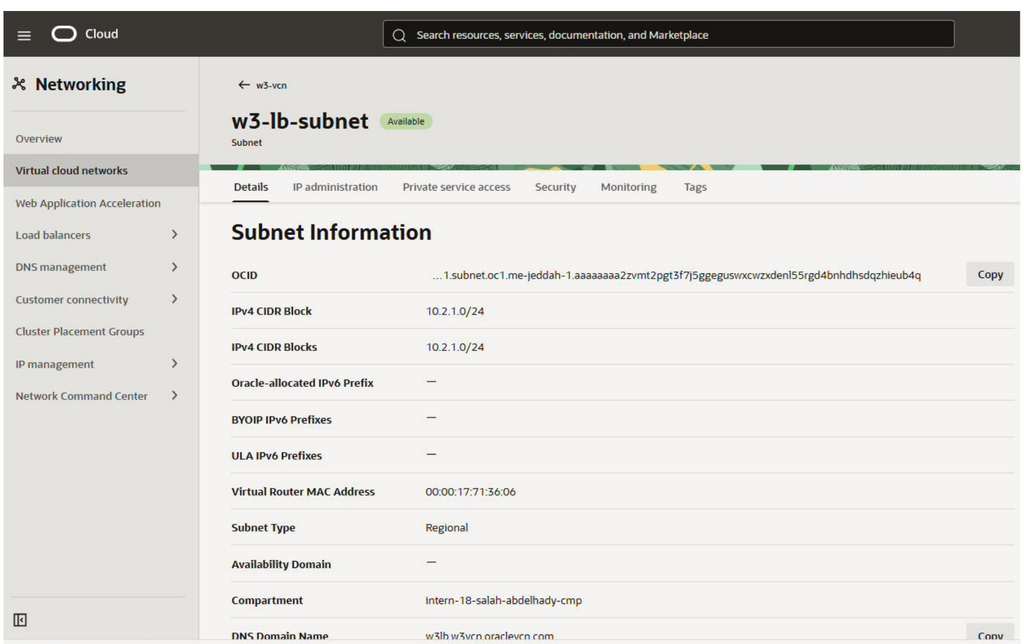


A02* — *NAT Gateway w3-nat Available.

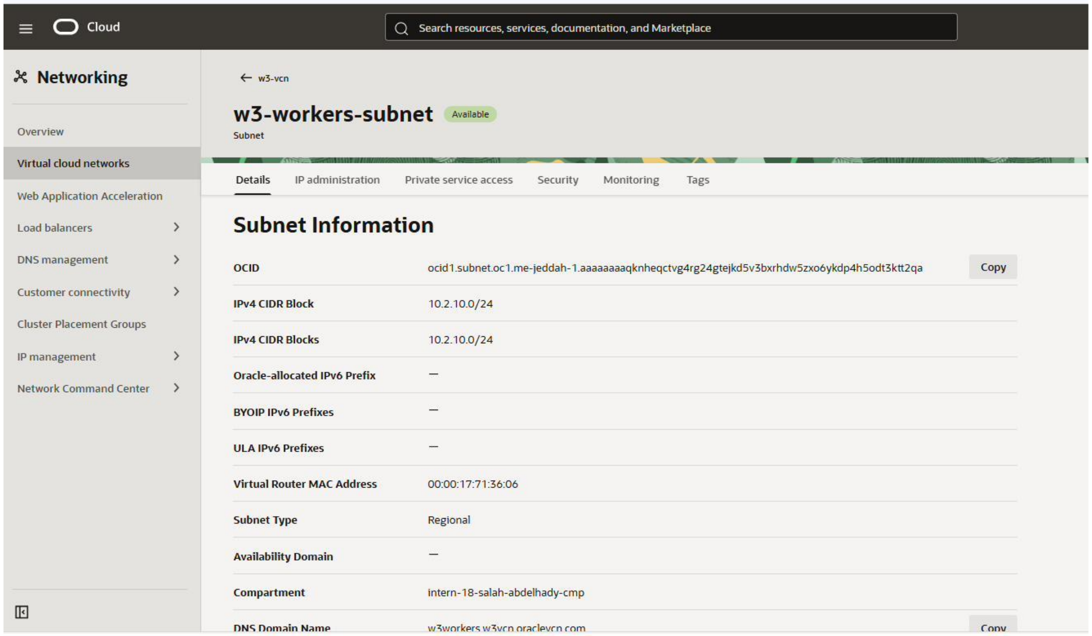


A02* — *Service Gateway w3-sgw Available, routed to All JED Services In Oracle Services Network.

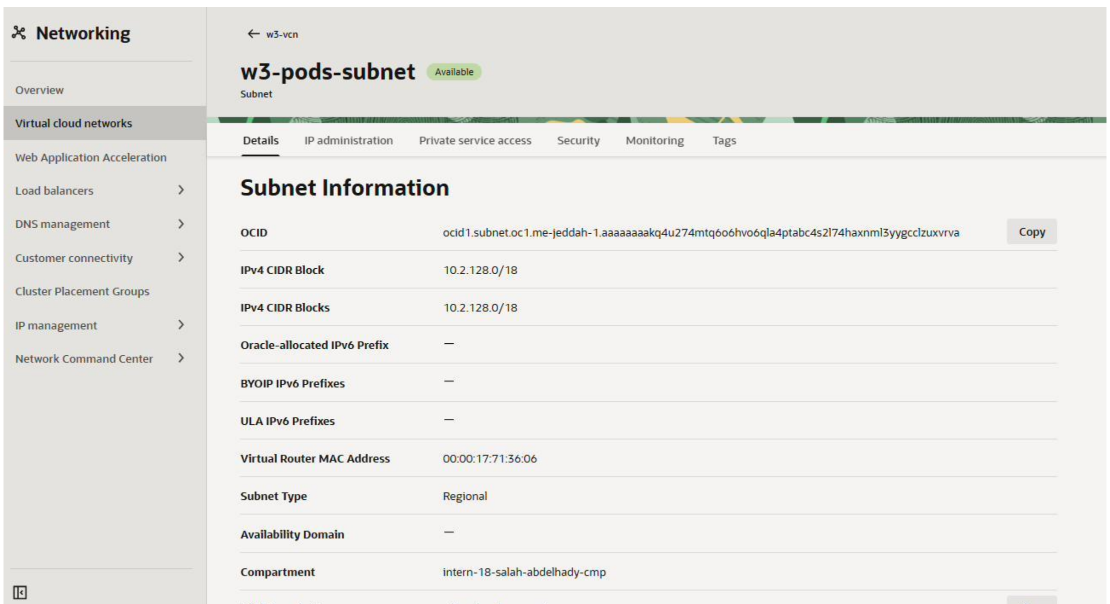
A03–A05 — subnets



A03* — *w3-lb-subnet, public, 10.2.1.0/24.

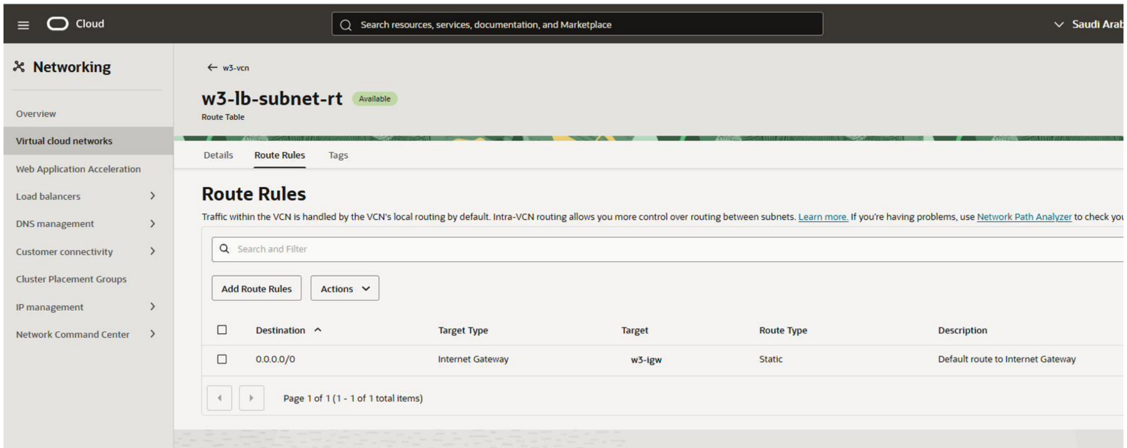


***A04** — w3-workers-subnet, private, 10.2.10.0/24.*

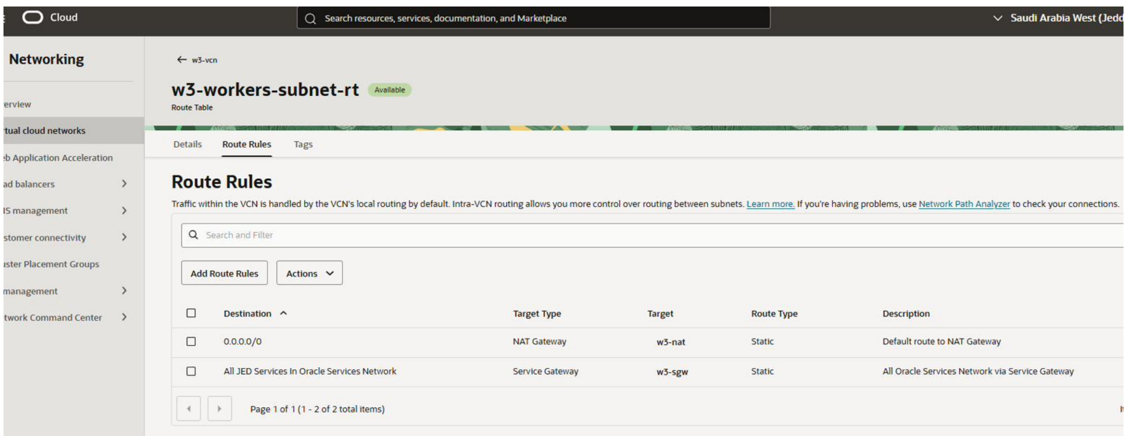


***A05** — w3-pods-subnet, private, 10.2.128.0/18.*

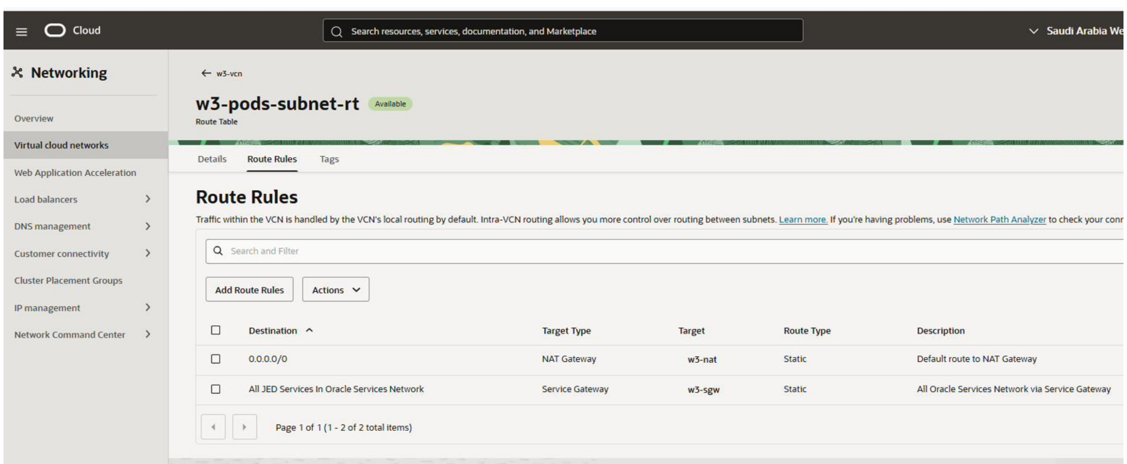
A06–A08 — route tables



A06 — w3-lb-subnet-rt — default route 0.0.0.0/0 to Internet Gateway w3-igw.

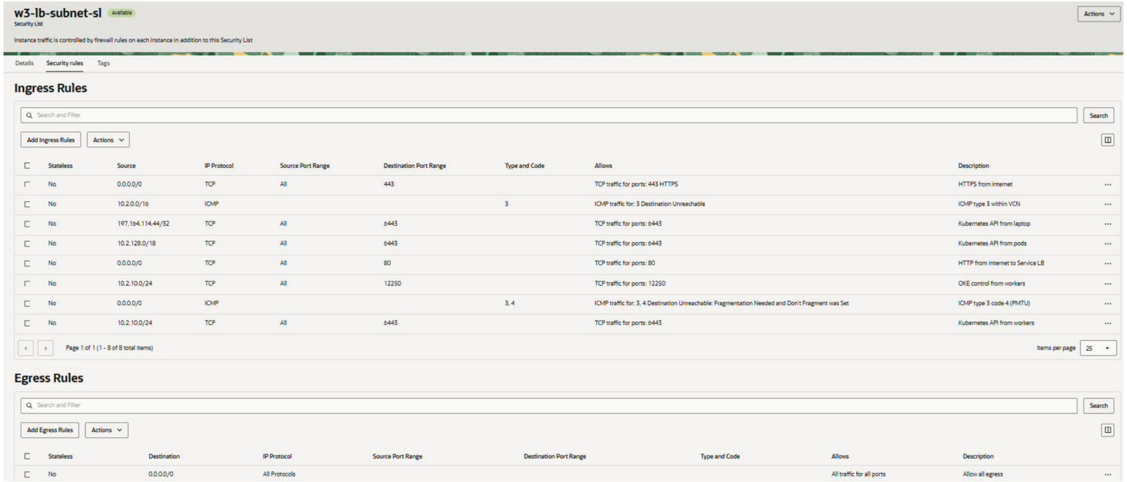


A07 — w3-workers-subnet-rt — routes through NAT w3-nat and Service Gateway w3-sgw.



A08 — w3-pods-subnet-rt — routes through NAT w3-nat and Service Gateway w3-sgw.

A09–A11 — security lists



w3-lb-subnet-sl Available
Security List
Instance traffic is controlled by firewall rules on each Instance in addition to this Security List

Details | Security rules | Tags

Ingress Rules

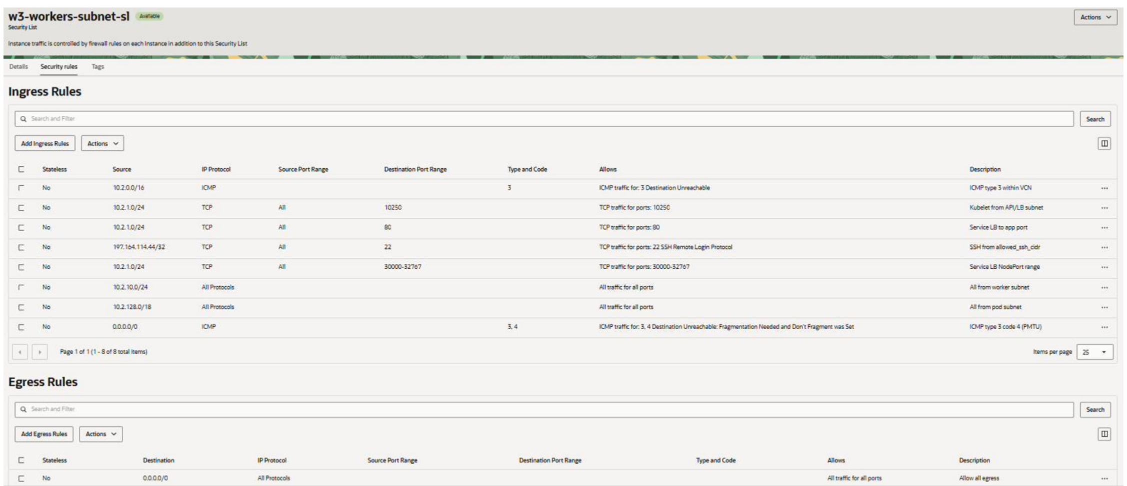
Stateless	Source	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
No	0.0.0.0/0	TCP	All	443		TCP traffic for ports: 443 HTTPS	HTTPS from internet
No	10.2.0.0/16	ICMP			3	ICMP traffic for: 3 Destination Unreachable	ICMP type 3 within VCN
No	197.164.114.44/32	TCP	All	6443		TCP traffic for ports: 6443	Kubernetes API from laptop
No	10.2.128.0/18	TCP	All	6443		TCP traffic for ports: 6443	Kubernetes API from pods
No	0.0.0.0/0	TCP	All	80		TCP traffic for ports: 80	HTTP from internet to Service LB
No	10.2.10.0/24	TCP	All	12250		TCP traffic for ports: 12250	OKE control from workers
No	0.0.0.0/0	ICMP			3, 4	ICMP traffic for: 3, 4 Destination Unreachable: Fragmentation Needed and Don't Fragment was Set	ICMP type 3 code 4 (PMTU)
No	10.2.10.0/24	TCP	All	6443		TCP traffic for ports: 6443	Kubernetes API from workers

Page 1 of 1 (1 - 8 of 8 total items)

Egress Rules

Stateless	Destination	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
No	0.0.0.0/0	All Protocols				All traffic for all ports	Allow all egress

A09* — *w3-lb-subnet-sl — HTTP/HTTPS plus the Kubernetes API ports 6443 and 12250.



w3-workers-subnet-sl Available
Security List
Instance traffic is controlled by firewall rules on each Instance in addition to this Security List

Details | Security rules | Tags

Ingress Rules

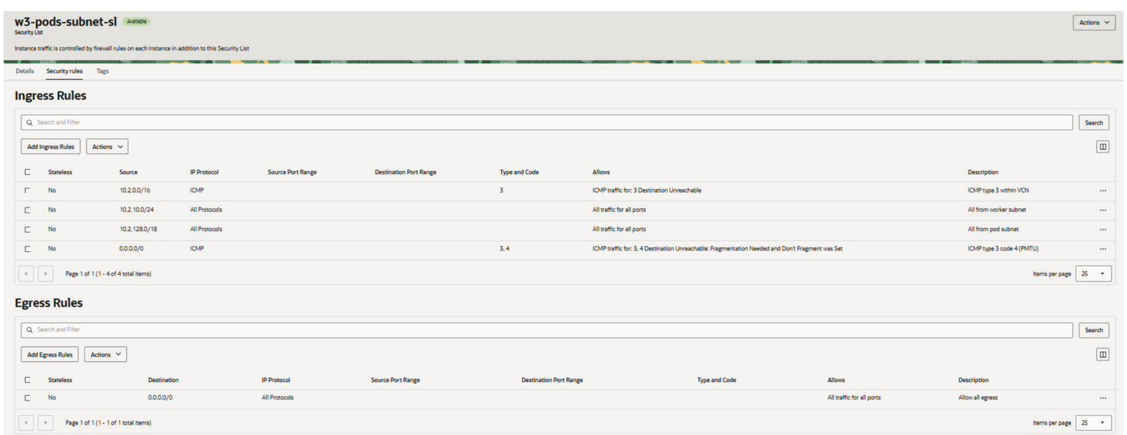
Stateless	Source	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
No	10.2.0.0/16	ICMP			3	ICMP traffic for: 3 Destination Unreachable	ICMP type 3 within VCN
No	10.2.1.0/24	TCP	All	10250		TCP traffic for ports: 10250	Kubelet from API/LB subnet
No	10.2.1.0/24	TCP	All	80		TCP traffic for ports: 80	Service LB to app port
No	197.164.114.44/32	TCP	All	22		TCP traffic for ports: 22 SSH Remote Login Protocol	SSH from allowed_ssh_cidr
No	10.2.1.0/24	TCP	All	30000-32767		TCP traffic for ports: 30000-32767	Service LB NodePort range
No	10.2.10.0/24	All Protocols				All traffic for all ports	All from worker subnet
No	10.2.128.0/18	All Protocols				All traffic for all ports	All from pod subnet
No	0.0.0.0/0	ICMP			3, 4	ICMP traffic for: 3, 4 Destination Unreachable: Fragmentation Needed and Don't Fragment was Set	ICMP type 3 code 4 (PMTU)

Page 1 of 1 (1 - 8 of 8 total items)

Egress Rules

Stateless	Destination	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
No	0.0.0.0/0	All Protocols				All traffic for all ports	Allow all egress

A10* — *w3-workers-subnet-sl — kubelet 10250, application port 80, NodePort range, and SSH.



w3-pods-subnet-sl Available
Security List
Instance traffic is controlled by firewall rules on each Instance in addition to this Security List

Details | Security rules | Tags

Ingress Rules

Stateless	Source	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
No	10.2.0.0/16	ICMP			3	ICMP traffic for: 3 Destination Unreachable	ICMP type 3 within VCN
No	10.2.10.0/24	All Protocols				All traffic for all ports	All from worker subnet
No	10.2.128.0/18	All Protocols				All traffic for all ports	All from pod subnet
No	0.0.0.0/0	ICMP			3, 4	ICMP traffic for: 3, 4 Destination Unreachable: Fragmentation Needed and Don't Fragment was Set	ICMP type 3 code 4 (PMTU)

Page 1 of 1 (1 - 4 of 4 total items)

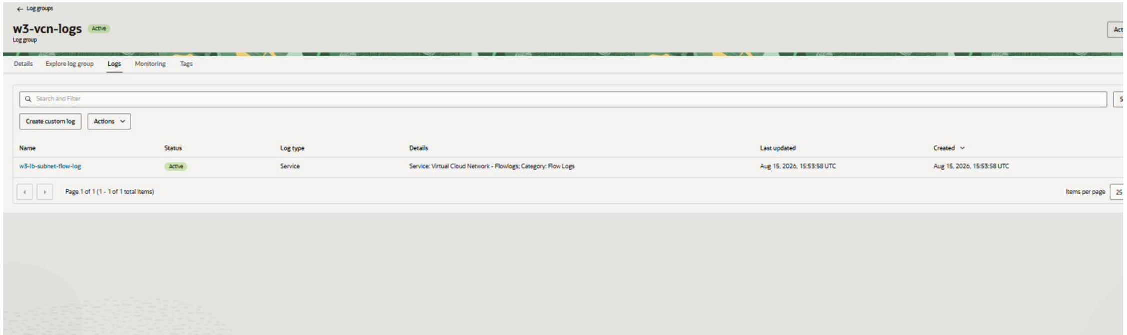
Egress Rules

Stateless	Destination	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
No	0.0.0.0/0	All Protocols				All traffic for all ports	Allow all egress

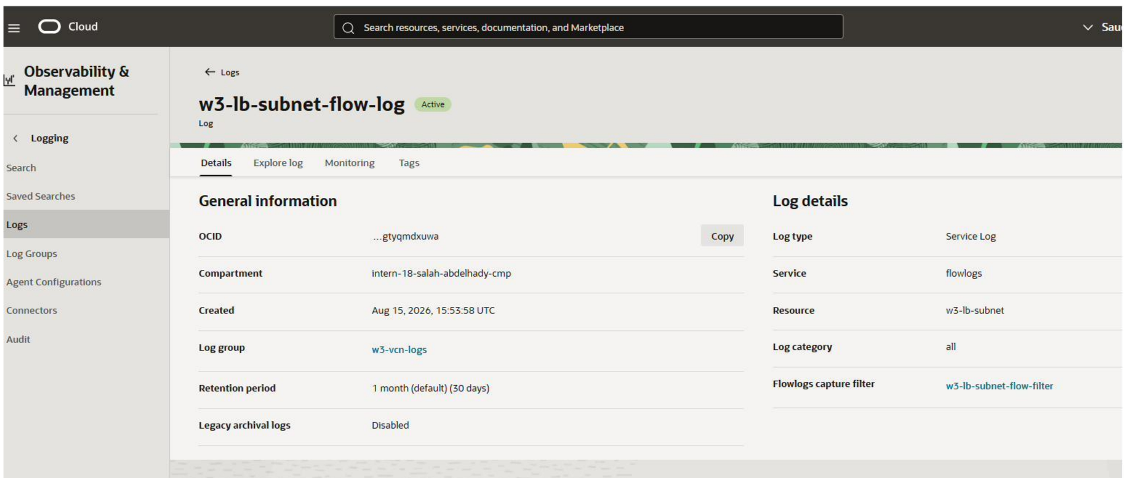
Page 1 of 1 (1 - 1 of 1 total items)

A11* — *w3-pods-subnet-sl — all traffic from the workers and pods subnets.

A12–A13 — logging

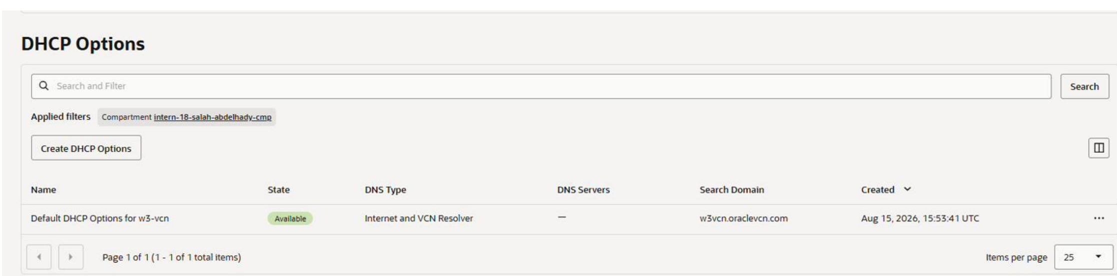


A12 — *Log group w3-vcn-logs, Active.*



A13 — *Flow log w3-lb-subnet-flow-log, Active on w3-lb-subnet.*

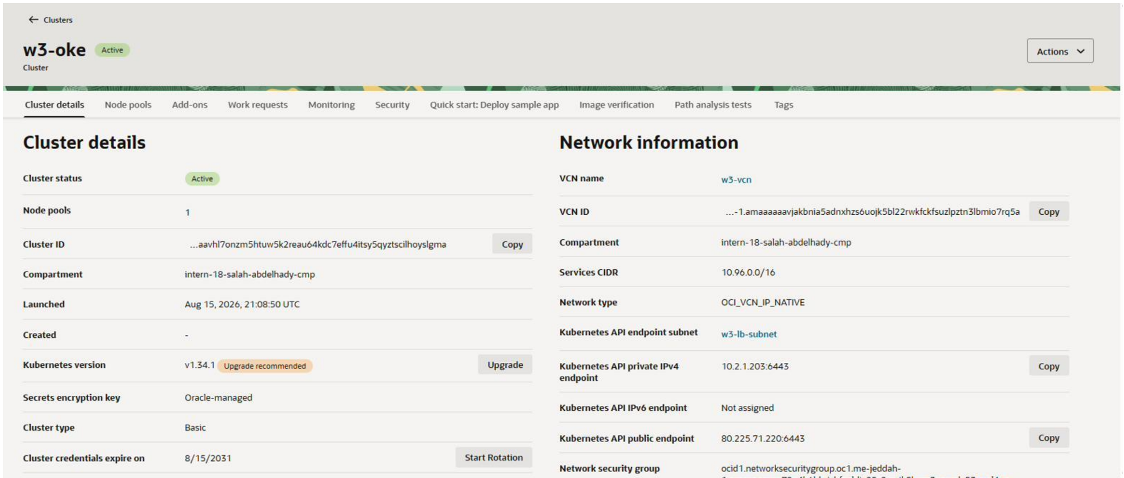
A99 — DHCP options



A99 — *Default DHCP options for w3-vcn.*

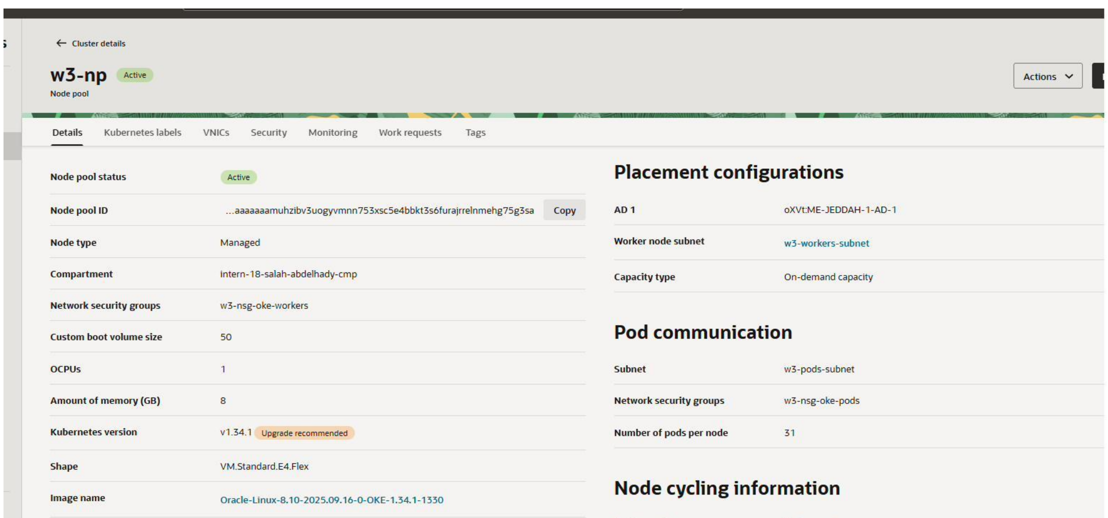
9.2 Stage B — OKE

B01 — cluster detail



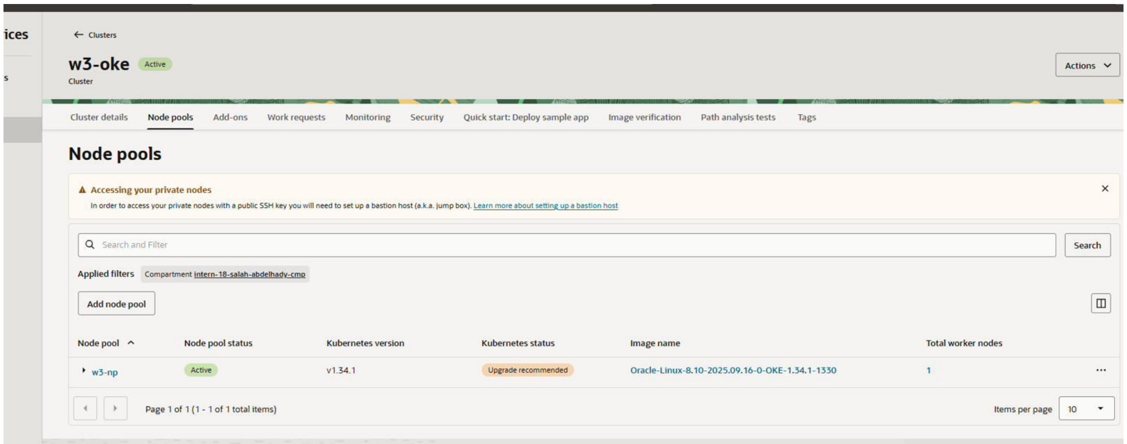
__B01__ — Cluster w3-oke Active, network type OCI_VCN_IP_NATIVE, Kubernetes v1.34.1.

B02 — node pool



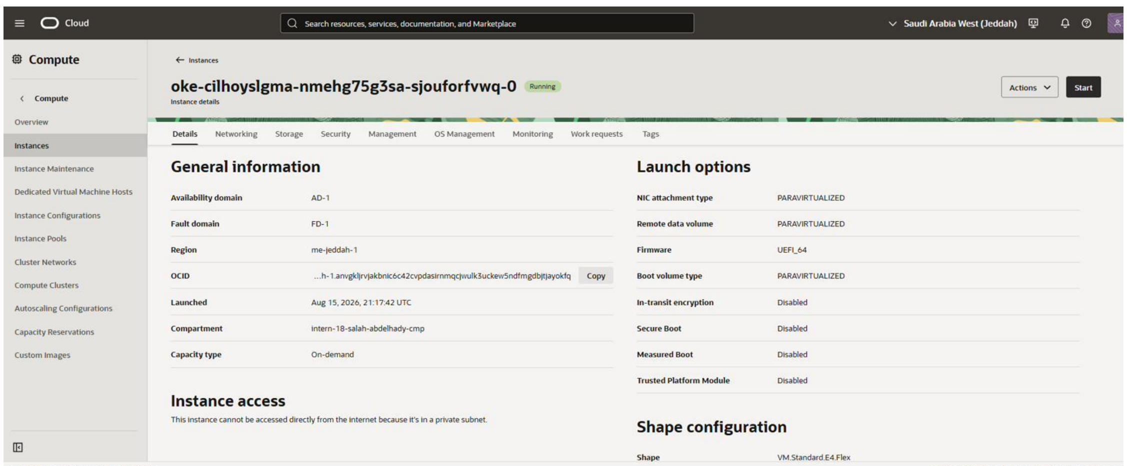
__B02__ — Node pool w3-np Active, shape VM.Standard.E4.Flex (1 OCPU / 8 GB), with the worker and pod subnets and NSGs attached.

B02b — node pools tab



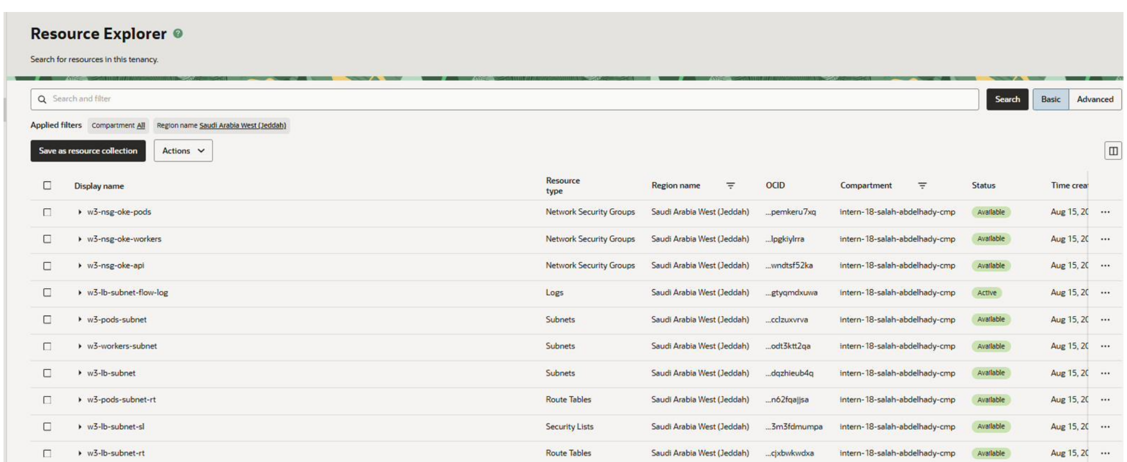
B02b — Cluster Node pools tab: w3-np Active, one worker.

B04 — worker instance



B04 — Worker instance Running in the private subnet only, AD-1 in me-jeddah-1, private IP 10.2.10.193.

B05 — OKE NSGs



B05 — NSGs w3-nsg-oke-api, w3-nsg-oke-workers, and w3-nsg-oke-pods, all Available.

9.3 Stage C — workload and destroy

C01 — pods and deployment

```
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke> $env:KUBECONFIG = "$env:USERPROFILE\.kube\w3-oke"
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke> kubectl -n w3-demo get pods,deploy
NAME                          READY   STATUS    RESTARTS   AGE
pod/nginx-58749cf6cc-hp8rz    1/1     Running   0          7m20s

NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx   1/1     1            1           10m
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke>
```

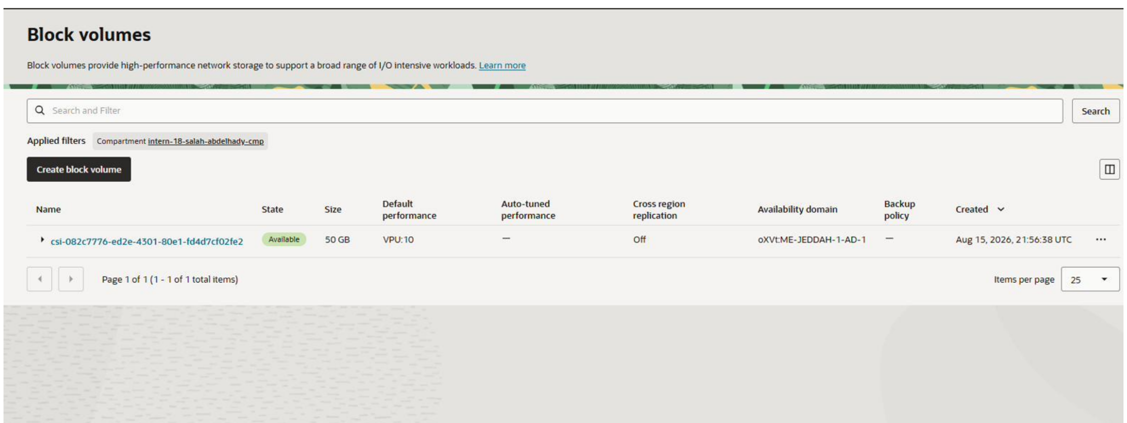
***C01** — kubectl -n w3-demo get pods,deploy — pod Running 1/1, deployment Available.*

C02 — PVC bound

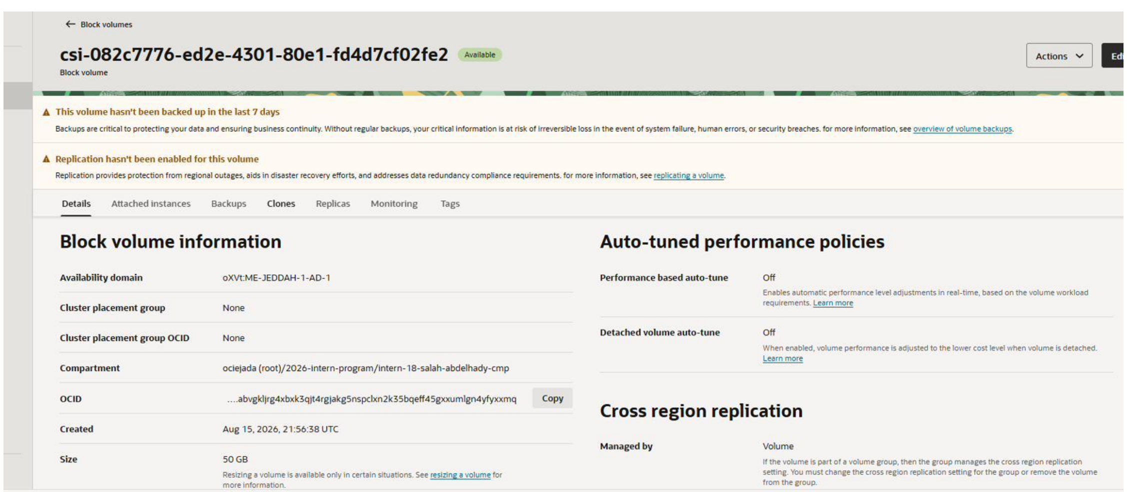
```
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke> kubectl -n w3-demo get pvc nginx-data
NAME         STATUS   VOLUME                                       CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AG
E
nginx-data   Bound    csi-082c7776-ed2e-4301-80e1-fd4d7cf02fe2    50Gi       RWO            oci-bv         <unset>                 11
m
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke>
```

***C02** — PVC nginx-data Bound, 50Gi, StorageClass oci-bv.*

C03 — block volume



***C03** — Block volumes list: the CSI-provisioned volume, 50 GB, Available.*



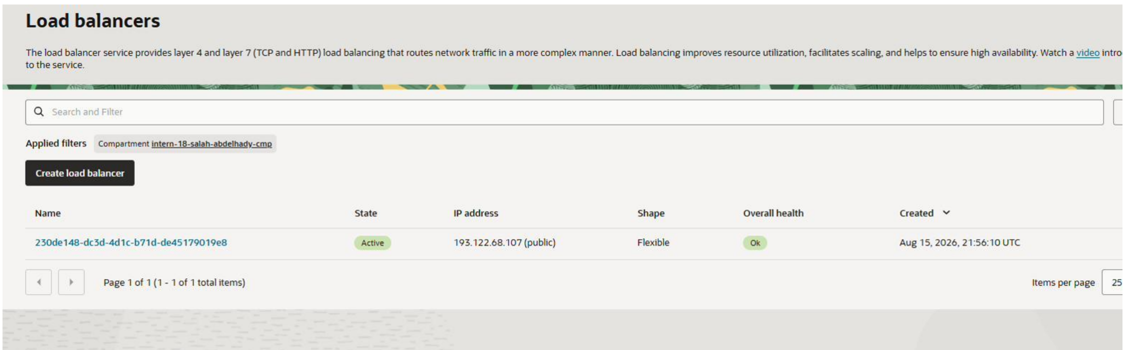
***C03b** — CSI volume detail (csi-082c7776-…), 50 GB, in the intern-18 compartment.*

C04 — Service external IP

```
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke> kubectl -n w3-demo get svc nginx
NAME    TYPE           CLUSTER-IP     EXTERNAL-IP      PORT(S)        AGE
nginx   LoadBalancer   10.96.15.207   193.122.68.107   80:31785/TCP   13m
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke>
```

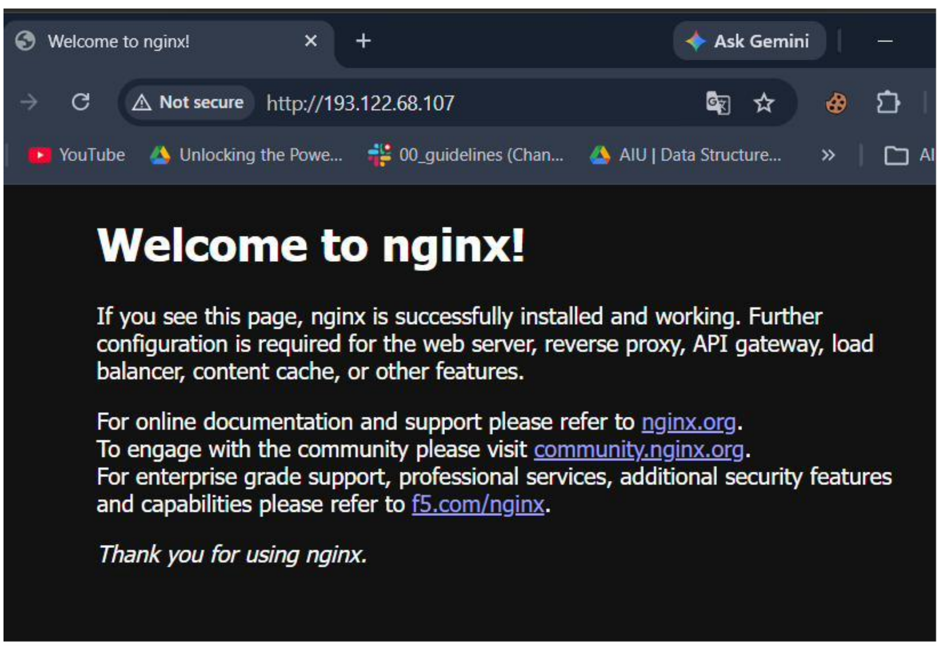
C04 — kubectl -n w3-demo get svc nginx — external IP 193.122.68.107.

C05 — console load balancer



C05 — Load balancers list: the Flexible load balancer, public IP 193.122.68.107, health Ok.

C06 — browser check



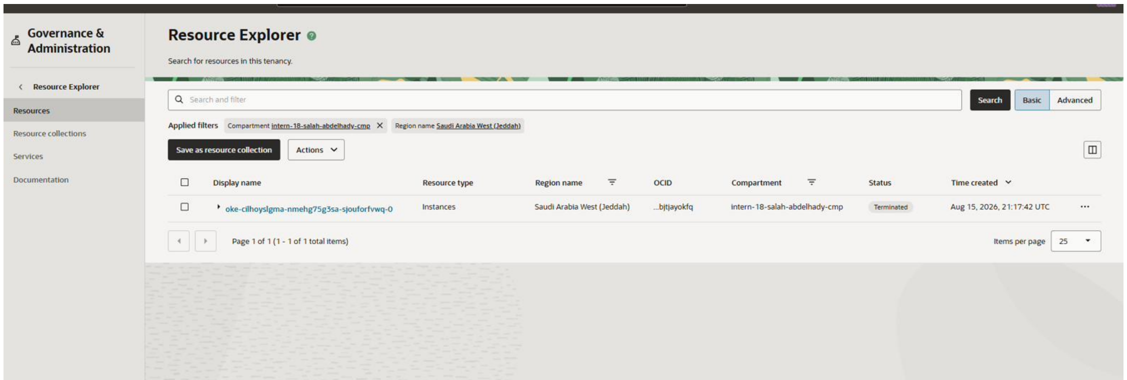
C06 — Browser at http://193.122.68.107/ showing the nginx welcome page.

C07 — volume mount

```
nginx   LoadBalancer   10.96.15.207   193.122.68.107   80:31785/TCP   13m
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke> kubectl -n w3-demo exec deploy/nginx -c nginx -- df -h /data
Filesystem                Size      Used Available Use% Mounted on
/dev/sdb                 48.9G     24.0K     48.9G   0% /data
PS E:\work\Ejada\terraform\nonprd\week3-containerized-oke>
```

C07 — kubectl exec … -- df -h /data — /dev/sdb mounted on /data.

C08 — post-destroy inventory



C08 — Resource Explorer after destroy: only a TERMINATED OKE worker ghost remains.

10. Issues and Troubleshooting

Both issues below were resolved before the destroy step, and neither left an open follow-up: the stack was destroyed cleanly once evidence capture was complete.

10.1 Cluster creation was rejected with a 404 authorization error

Observation

The first apply with enable_oke = true failed on the cluster create call, returning 404-NotAuthorizedOrNotFound. The node pool was never attempted because Terraform stopped at the failed resource.

```
Error: 404-NotAuthorizedOrNotFound, Authorization failed or requested resource not found
  on oke.tf line 1, in resource "oci_containerengine_cluster" "this":
   1: resource "oci_containerengine_cluster" "this" {
```

Investigation

- Re-checked the compartment target in the error output against intern-18-salah-abdelhady-cmp — the compartment itself was correct, ruling out a simple typo or wrong-OCID mistake.
- Re-ran terraform plan against the existing state to confirm the failure was isolated to the cluster create call and not a wider provider or network issue — the network module's resources (VCN, gateways, subnets) all planned clean with no changes.
- Reviewed the intern IAM policy statements available at the time and found no explicit grant for the cluster-family resource family or the OKE service, which narrowed the cause to a missing policy statement rather than a Terraform configuration defect.

Resolution

Requested a policy update from the mentoring team to add create permission on cluster-family for the OKE service in the intern-18 compartment. No changes were needed on the Terraform side — the module and variables were already correct for the intended cluster shape.

Verification

Once the policy statement was in place, terraform apply was re-run unchanged and completed in full: cluster w3-oke reached ACTIVE in roughly 8m40s and node pool w3-np followed in roughly 5m26s. terraform state list and the OCI CLI snapshot in Section 8.1 confirm both resources as ACTIVE with CNI OCI_VCN_IP_NATIVE.

10.2 Reapplying the storage class conflicted with the cluster's built-in definition

Observation

kubectl apply -f k8s/storageclass.yaml was rejected because OKE's control plane already provisions the oci-bv StorageClass at cluster creation, and the manifest's apply attempted to update fields the API treats as immutable on an existing StorageClass.

Investigation

- Ran kubectl get storageclass and confirmed oci-bv already existed with the same provisioner (blockvolume.csi.oraclecloud.com) that the manifest declared.
- Compared the manifest's parameters against the live object and confirmed the intent was identical — the manifest was documenting the class the cluster already exposes, not introducing a new one.

Resolution

Skipped the redundant apply of storageclass.yaml and proceeded directly to pvc.yaml, which references storageClassName: oci-bv by name. The manifest remains in the repository as the explicit, version-controlled definition of the class the workload depends on.

Verification

kubectl -n w3-demo get pvc nginx-data showed Bound against the existing oci-bv class within seconds of applying the PVC, and the volume mounted successfully inside the pod (Section 7.4, Screenshot C02 and C07).

11. Cleanup and Destroy Verification

1. Validated pods, PVC binding, Service external IP, the browser check, and the /data mount while the stack was live.
2. Deleted the Kubernetes Service, Deployment, PVC, and namespace.
3. Ran terraform destroy -auto-approve, which completed with 36 resources destroyed at exit code 0.
4. Confirmed terraform state list returned empty.
5. Ran an OCI CLI post-destroy inventory across me-jeddah-1 for the intern-18 compartment.

Family	Residual live?
VCN / subnets / Internet Gateway / NAT / Service Gateway	No
NSGs / route tables / security lists	No
OKE clusters	No (DELETED list-API ghost only)
Load balancers	No
Block / boot volumes	No (TERMINATED CSI and boot ghosts only)
Instances	No RUNNING instances — TERMINATED OKE worker only
Log groups	No

The compartment is lab-empty for billable Week 3 resources. The intern-18 compartment object itself was not deleted, consistent with the shared, long-lived nature of the intern compartment across weekly labs. Console confirmation: Screenshot C08 (Section 9.3).

12. Document Index

This report is the complete Week 3 Lab 3 submission package. It contains, in one file:

- Sections 1–6 — objectives, environment, architecture, source inventory, variables, and design standards, with Figures 1–5.
- Sections 7–8 — the full command history for the successful path and the evidence catalogue backing it.
- Section 9 — the curated Console and kubectl screenshot set (31 captures) organized by stage.
- Sections 10–11 — the two issues encountered and how each was diagnosed and resolved, and the destroy verification.

Working stack (source-controlled): github.com/Salahmohameds/ejada-oci-week1-terraform

End of Terraform documentation for Week 3 Lab 3 — covering code, commands, evidence, diagrams, and the destroy.